

Homework 3

Ravi Kini

March 13, 2026

Exercise 1

Part (a)

Let $C \in \mathbb{C}^{n \times n}$ be a circulant matrix, with $C_{j,k} = c_{(j-k) \bmod n}$. The $n \times n$ discrete Fourier transform matrix $\mathcal{F}_n \in \mathbb{C}^{n \times n}$ is such that $(\mathcal{F}_n)_{j,k} = \omega^{jk}$ where $\omega = e^{-2\pi i/n}$. Consider the k -th column of $\mathcal{F}_n$, $(\mathcal{F}_n)_{\cdot,k}$. We then see that:

$$(C(\mathcal{F}_n)_{\cdot,k})_j = \sum_{l=0}^{n-1} C_{j,l}(\mathcal{F}_n)_{l,k} = \sum_{l=0}^{n-1} C_{j,l}\omega^{lk}$$

Letting $m = (j - l) \bmod n$:

$$(C(\mathcal{F}_n)_{\cdot,k})_j = \sum_{m=0}^{n-1} c_m \omega^{(j-m)k} = \omega^{jk} \sum_{m=0}^{n-1} c_m \omega^{-mk} = (\mathcal{F}_n)_{j,k} \sum_{m=0}^{n-1} c_m \omega^{-mk}$$

It follows that $(C(\mathcal{F}_n)_{\cdot,k})_j = \left(\sum_{m=0}^{n-1} c_m \omega^{-mk} \right) (\mathcal{F}_n)_{j,k}$, and so that the k -th column of the discrete Fourier transform matrix is an eigenvector of C .

Part (b)

As seen in the previous part, the columns of the discrete Fourier transform matrix are the eigenvectors of C , with column k having the corresponding eigenvalue $\sum_{m=0}^{n-1} c_m \omega^{-mk}$. Letting c be the first column of C , we see that:

$$(\mathcal{F}_n c)_k = \sum_{m=0}^{n-1} (\mathcal{F}_n)_{k,m} c_m = \sum_{m=0}^{n-1} \omega^{-km} c_m$$

This quantity is identical to the eigenvalues found in the previous part.

Part (c)

To compute the n eigenvalues of C , we only need to compute $\mathcal{F}_n c$. Naively, this would require $O(n^2)$ floating point operations. However, multiplication using FFT requires only $O(n \log n)$ floating point operations. By exploiting the existing structure of $\mathcal{F}_n$, computation can be sped up considerably.

Part (d)

Let T be a Toeplitz matrix with first column $(c_0, c_1, \dots, c_{n-1})$ and first row $(c_0, r_1, \dots, r_{n-1})$. We then define the $2n \times 2n$ circulant matrix C with the first column $(c_0, c_1, \dots, c_{n-1}, 0, r_{n-1}, \dots, r_1)$. The upper left block of this new matrix is T . To see this, for j, k where $0 \leq j, k \leq n-1$, when $j \geq k$:

$$C_{j,k} = c_{j-k} = T_{j,k}$$

and when $j < k$:

$$C_{j,k} = r_{k-j} = T_{j,k}$$

To do $O(n \log n)$ matrix-vector product Tx given a $n \times n$ Toeplitz matrix T , we first construct C as described above and $y = (x_0, \dots, x_{n-1}, 0)$. Using the FFT, we can compute Cy using $O(n \log n)$ floating point operations; to find Tx , we only need to extract the first n entries of Cy .

Exercise 2

Part (a)

Let there be N source particles in the interval $D = [-\frac{1}{2}, \frac{1}{2}]$, with particle j at position s_j and with charge q_j . The potential at a target point outside D is:

$$\Phi(x) = \sum_{j=1}^N \frac{q_j}{x - s_j}$$

For x such that $|x| > |s_j|$, we see that:

$$\frac{1}{x - s_j} = \frac{1}{x} \cdot \frac{1}{1 - s_j/x} = \frac{1}{x} \sum_{k=0}^{\infty} \left(\frac{s_j}{x}\right)^k = \sum_{k=0}^{\infty} \frac{s_j^k}{x^{k+1}}$$

Consequently:

$$\begin{aligned} \Phi(x) &= \sum_{j=1}^N q_j \sum_{k=0}^{\infty} \frac{s_j^k}{x^{k+1}} \\ &= \sum_{k=0}^{\infty} \frac{\sum_{j=1}^N q_j s_j^k}{x^{k+1}} = \sum_{k=0}^{\infty} \frac{M_k}{x^{k+1}} \end{aligned}$$

where $M_k = \sum_{j=1}^N q_j s_j^k$.

Part (b)

Let $\Phi_p(x) = \sum_{k=0}^{p-1} \frac{M_k}{x^{k+1}}$ be the p -term approximation of the potential. Let $Q = \sum_{j=1}^N |q_j|$ and $s_{\max} = \max |s_j|$. We see that

$$|M_k| = \left| \sum_{j=1}^N q_j s_j^k \right| \leq \sum_{j=1}^N |q_j| |s_j|^k \leq Q s_{\max}^k$$

Then the error term is bounded by:

$$\begin{aligned} |\Phi(x) - \Phi_p(x)| &= \left| \sum_{k=p}^{\infty} \frac{M_k}{x^{k+1}} \right| \leq \sum_{k=p}^{\infty} \frac{|M_k|}{|x|^{k+1}} \\ &\leq \sum_{k=p}^{\infty} \frac{Q s_{\max}^k}{|x|^{k+1}} = \frac{Q}{|x|} \sum_{k=p}^{\infty} \left(\frac{s_{\max}}{|x|} \right)^k \\ &\leq \frac{Q}{|x|} \cdot \frac{(s_{\max}/|x|)^p}{1 - s_{\max}/|x|} = \frac{Q}{|x| - s_{\max}} \left(\frac{s_{\max}}{|x|} \right)^p \end{aligned}$$

Part (c)

We seek $\tilde{M}_l$ such that $\Phi(x) = \sum_{l=0}^{\infty} \frac{\tilde{M}_l}{(x-x_c)^{l+1}}$. We first note that:

$$\begin{aligned} \frac{1}{x-s_j} &= \frac{1}{(x-x_c) + (x_c-s_j)} = \frac{1}{x-x_c} \cdot \frac{1}{1 + \frac{x_c-s_j}{x-x_c}} \\ &= \frac{1}{x-x_c} \sum_{l=0}^{\infty} \left(-\frac{x_c-s_j}{x-x_c} \right)^l \\ &= \sum_{l=0}^{\infty} \frac{(-1)^l (x_c-s_j)^l}{(x-x_c)^{l+1}} \end{aligned}$$

Consequently, substituting into the formula for the potential:

$$\begin{aligned} \Phi(x) &= \sum_{j=1}^N q_j \sum_{l=0}^{\infty} \frac{(-1)^l (x_c-s_j)^l}{(x-x_c)^{l+1}} \\ &= \sum_{l=0}^{\infty} \frac{\sum_{j=1}^N q_j (-1)^l (x_c-s_j)^l}{(x-x_c)^{l+1}} = \sum_{l=0}^{\infty} \frac{\tilde{M}_l}{(x-x_c)^{l+1}} \end{aligned}$$

where $\tilde{M}_l = \sum_{j=1}^N q_j (-1)^l (x_c-s_j)^l$. To see the relationship between $\tilde{M}_l$ and M_k :

$$\begin{aligned} \tilde{M}_l &= \sum_{j=1}^N q_j (-1)^l (x_c-s_j)^l \\ &= \sum_{j=1}^N q_j (-1)^l \sum_{k=0}^l \binom{l}{k} x_c^{l-k} (-s_j)^k \\ &= \sum_{k=0}^l \binom{l}{k} 2^{l-k} (-1)^{l+k} M_k \end{aligned}$$

If $x_c = 2$:

$$\tilde{M}_l = \sum_{k=0}^l \binom{l}{k} 2^{l-k} (-1)^{l+k} M_k$$

Part (d)

Let $\mu = [M_0, \dots, M_{p-1}]^T$ and $\tilde{\mu} = [\tilde{M}_0, \dots, \tilde{M}_{p-1}]^T$. The $p \times p$ translation operator T transforms the vector of old moments to the vector of new moments, with $\tilde{\mu} = T\mu$; we see that it has entries such that:

$$T_{l,k} = \binom{l}{k} x_c^{l-k} (-1)^{l+k}$$

Part (e)

For all target points such that $|x| > 10$, for the truncation error to be at most 10^{-6} :

$$\begin{aligned} 10^{-6} &\leq \frac{Q}{10 - 0.5} \left(\frac{0.5}{10} \right)^p \\ -6 &\leq \log(Q/9.5) + p \log(0.05) \\ p &\geq \frac{-6 - \log(Q/9.5)}{\log(0.05)} \end{aligned}$$

The bound is highest when $Q = 1$ (there is only a single charge), which yields an lower bound of $p \geq 3.86$; at least 4 terms are then needed in the approximation.

Exercise 3

Part (a)

Let $A \in \mathbb{C}^{n \times n}$ be a diagonalizable matrix with diagonalization $A = V\Lambda V^{-1}$. We apply GMRES to solve $Ax = b$, with initial guess $x_0 \in \mathbb{C}^n$. Let $r_k = b - Ax_k$. On the k th iteration, GMRES will find the iterate $x_k \in x_0 + \mathcal{K}_k(A, r_0)$ that minimizes $\|r_k\|_2$. We can write $x_k - x_0 = q_{k-1}(A)r_0$ where q_{k-1} is a polynomial of degree at most $k-1$; it then follows that:

$$\begin{aligned} r_k &= b - Ax_k = b - A(x_0 + q_{k-1}(A)r_0) \\ &= r_0 - q_{k-1}(A)r_0 \\ &= (I - Aq_{k-1}(A))r_0 = p_k(A)r_0 \end{aligned}$$

where $p_k(\lambda) = 1 - \lambda q_{k-1}(\lambda)$. Note that it is a polynomial of degree k such that $p_k(0) = 1$. It is then equivalent to state that GMRES finds x_k that minimizes $\|r_k\|_2 = \min_{p \in \mathcal{P}_k, p(0)=1} \|p(A)r_0\|_2$. We then see that:

$$\begin{aligned} \|p_k(A)r_0\|_2 &= \|Vp(\Lambda)V^{-1}r_0\|_2 \\ &\leq \|V\|_2 \cdot \|p(\Lambda)\|_2 \cdot \|V^{-1}\|_2 \cdot \|r_0\|_2 \\ &\leq \kappa(V)\|r_0\|_2 \cdot \|p(\Lambda)\|_2 \end{aligned}$$

Note that since Λ is a diagonal matrix, $p(\Lambda) = \text{diag}(p(\lambda_1), \dots, p(\lambda_n))$, and so $\|p(\Lambda)\|_2 = \max_i |p(\lambda_i)|$. Consequently:

$$\begin{aligned} \|r_k\|_2 &\leq \min_{p \in \mathcal{P}_k, p(0)=1} \kappa(V)\|r_0\|_2 \cdot \max_i |p(\lambda_i)| \\ &\leq \min_{p \in \mathcal{P}_k, p(0)=1} \kappa(V)\|r_0\|_2 \max_{\lambda \in \sigma(A)} |p(\lambda)| \\ \frac{\|r_k\|_2}{\|r_0\|_2} &\leq \kappa(V) \min_{p \in \mathcal{P}_k, p(0)=1} \max_{\lambda \in \sigma(A)} |p(\lambda)| \end{aligned}$$

Part (b)

Consider the $n \times n$ matrix:

$$A = \begin{bmatrix} 0 & 0 & \dots & 0 & 1 \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 0 \end{bmatrix}$$

Suppose we wish to solve $Ax = b$ where $b = (1, 0, \dots, 0)^T$ using GMRES with $x_0 = 0$. Note that the solution is $x^* = e_n$. Since $r_0 = e_1$, $\mathcal{K}_k(A, r_0) = \text{span}\{e_1, Ae_1, \dots, A^{k-1}e_1\} = \text{span}\{e_1, e_2, \dots, e_k\}$. For all $k < n$, e_n then cannot be in the Krylov subspace, which means that it takes n iterations for GMRES to converge.

Part (c)

```

1 def zeros(n):
2     return [[0.0 for i in range(n)] for j in range(n)]
3
4 def m_v_prod(A, x):
5     n = len(A)
6     return [sum(A[i][j] * x[j] for j in range(n)) for i in range(n)]
7
8 def v_norm(x):
9     n = len(x)
10    return sum([x[j] ** 2 for j in range(n)]) ** 0.5
11
12 def v_add(x, y):
13     n = len(x)
14     return [x[j] + y[j] for j in range(n)]
15
16 def v_sub(x, y):
17     n = len(x)
18     return [x[j] - y[j] for j in range(n)]
19
20 def v_dot(x, y):
21     n = len(x)
22     return sum([x[j] * y[j] for j in range(n)])
23
24 def v_prod(x, k):
25     n = len(x)
26     return [x[j] * k for j in range(n)]
27
28 def givens(v1, v2):
29     t = (v1 ** 2 + v2 ** 2) ** 0.5
30     return v1 / t, v2 / t
31
32 def GMRES(A, b, x0, tol=1e-10, max_iter=1000):
33     n = len(b)
34     m = max_iter
35
36     b_norm = v_norm(b)
37     r0 = v_sub(b, m_v_prod(A, x0))
38     beta = v_norm(r0)
39
40     q = [[0.0] * n for _ in range(m + 2)]
41     q[1] = v_prod(r0, 1.0 / beta)
42
43     H = [[0.0] * (m + 1) for _ in range(m + 2)]
44     s = [0.0] * (m + 1)
45     c = [0.0] * (m + 1)
46     g = [0.0] * (m + 2)
47     g[1] = beta
48
49     for k in range(1, m + 1):
50         w = m_v_prod(A, q[k])
51
52         for j in range(1, k + 1):
53             H[j][k] = v_dot(w, q[j])
54             w = v_sub(w, v_prod(q[j], H[j][k]))
55
56         H[k + 1][k] = v_norm(w)
57         q[k + 1] = v_prod(w, 1.0 / H[k + 1][k])
58
59         for i in range(1, k):
60             H[i + 1][k], H[i][k] = -s[i] * H[i][k] + c[i] * H[i + 1][k], c[i] * H[i][k] + s[i] * H[i + 1][k]
61
62         c[k], s[k] = givens(H[k][k], H[k + 1][k])
63         H[k][k] = c[k] * H[k][k] + s[k] * H[k + 1][k]
64         H[k + 1][k] = 0.0

```

```

65
66     g[k + 1] = -s[k] * g[k]
67     g[k] = c[k] * g[k]
68
69     if abs(g[k + 1]) / b_norm <= tol:
70         break
71
72     alpha = [0.0] * (k + 1)
73     for i in range(k, 0, -1):
74         alpha[i] = g[i] - sum([H[i][j] * alpha[j] for j in range(i + 1, k + 1)])
75         alpha[i] /= H[i][i]
76
77     xk = x0[:]
78     for j in range(1, k + 1):
79         xk = v_add(xk, v_prod(q[j], alpha[j]))
80
81     return xk

```